this page intentionally empty

# 1 Experimental Design and Results

## 1.1 Results

Table 1 shows the total execution time of the various encodings to execute the benchmark ( `40` $fibonacci$ ) for integer and floating-point values on the Apple M1.

| Encoding | IntegerBr | Float |
|---|---:|---:|
| cachedPtr | 3220ms | 5113ms |
| compact | 3008ms | 2940ms |
| compact6 | 3000ms | 2990ms |
| compactI | 2900ms | 2991ms |
| compactI6 | 2884ms | 3047ms |
| nan | 3061ms | 3117ms |
| onlyFloat | N/A | 2609ms |
| onlyInt | 2663ms | N/A |
| ptr | 4852ms | 5195ms |
| spur | 2726ms | 3342ms |
| spurOpt | 2808ms | 3571ms |
| taggedHigh | 2940ms | 3684ms |
| taggedInt | 2726ms | 4829ms |
| taggedLow | 2889ms | 3698ms |
| taggedSMI | 2756ms | 4944ms |
| ultra | 2824ms | 3103ms |
| zagOrig | 2960ms | 3171ms |
| maxSpur | 2742ms | 3066ms |

**Table 1.** AArch64 (Apple M1) total timing for encodings

Figures 1 and 2 show at a glance the relative performance of the various encodings on the two architectures.

All timings are the median of 10 runs following 3 runs of warmups. They are run on Linux systems running in console mode. We calculate the mean, median and geometric mean for each example and they are essentially equal in all cases.

| Encoding | IntegerBr | Float |
|---|---:|---:|
| nan | 398ms | 508ms |
| maxSpur | 79ms | 457ms |
| ultra | 161ms | 494ms |
| spur | 63ms | 733ms |
| spurOpt | 145ms | 962ms |
| compact | 345ms | 331ms |
| compact6 | 337ms | 381ms |
| compactI | 237ms | 382ms |
| compactI6 | 221ms | 438ms |
| taggedLow | 226ms | 1089ms |
| taggedHigh | 277ms | 1075ms |
| taggedInt | 63ms | 2220ms |
| taggedSMI | 93ms | 2335ms |
| cachedPtr | 557ms | 2504ms |
| ptr | 2189ms | 2586ms |

**Table 2.** AArch64 (Apple M1) net timing for encodings

| Encoding | IntegerBr | Float |
|---|---:|---:|
| nan | 908ms | 854ms |
| maxSpur | 282ms | 1257ms |
| ultra | 302ms | 1029ms |
| spur | 445ms | 1302ms |
| spurOpt | 408ms | 1622ms |
| compact | 520ms | 638ms |
| compact6 | 522ms | 613ms |
| compactI | 341ms | 555ms |
| compactI6 | 348ms | 554ms |
| taggedLow | 442ms | 1163ms |
| taggedHigh | 767ms | 1162ms |
| taggedInt | 161ms | 3205ms |
| taggedSMI | 217ms | 3503ms |
| cachedPtr | 957ms | 3871ms |
| ptr | 3481ms | 4151ms |

**Table 3.** x86-64 (Linux) net timing for encodings

Tables 2 and 3 present the net execution times for each encoding across the two benchmark configurations on AArch64 (Apple M1) and x86-64 respectively. Each row reports the time for a single encoding under two conditions: **IntegerBr** (inlined branching integer Fibonacci), and **Float** (inlined float Fibonacci).

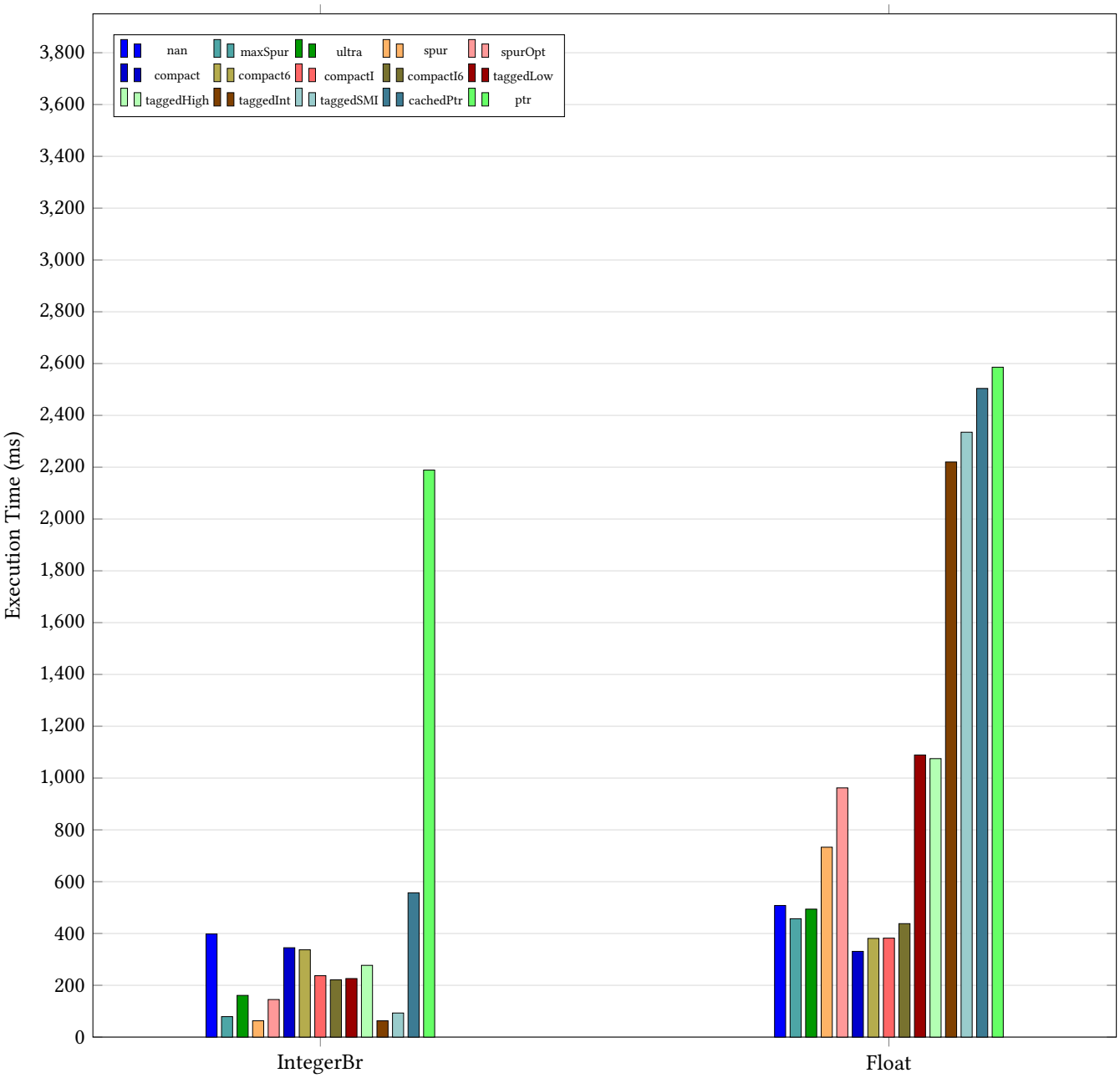


**Figure 1.** AArch64 benchmark results

### 1.1.1 Pareto Analysis.

The Pareto frontier contains the points that represent optimal trade-offs between two parameters. In this case, we are looking at integer versus floating-point performance in figure 5. The Pareto frontiers don't intersect, showing that the Aarch64 dominates the x86-64 in this experiment.

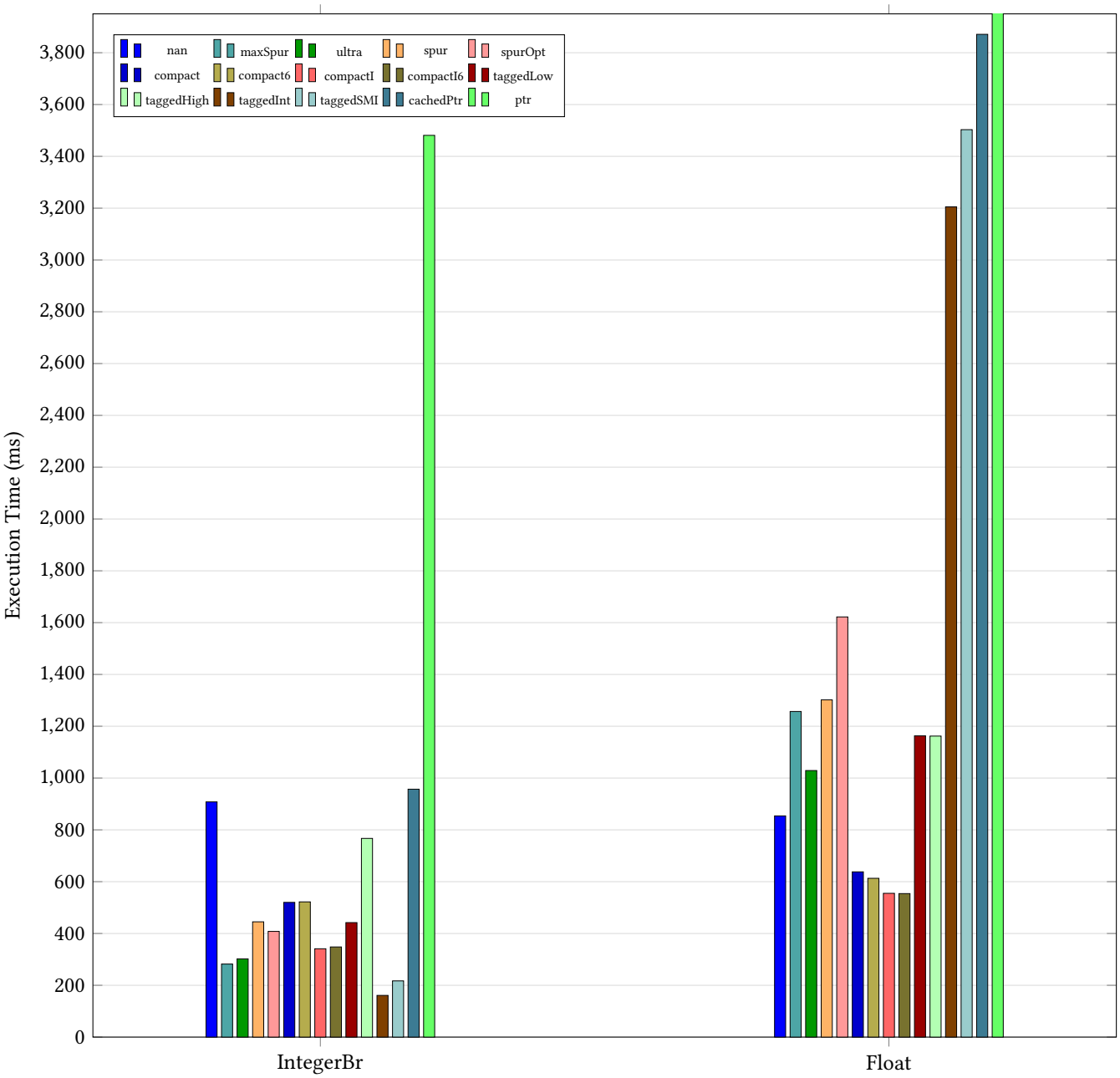


**Figure 2.** x86_64 benchmark results

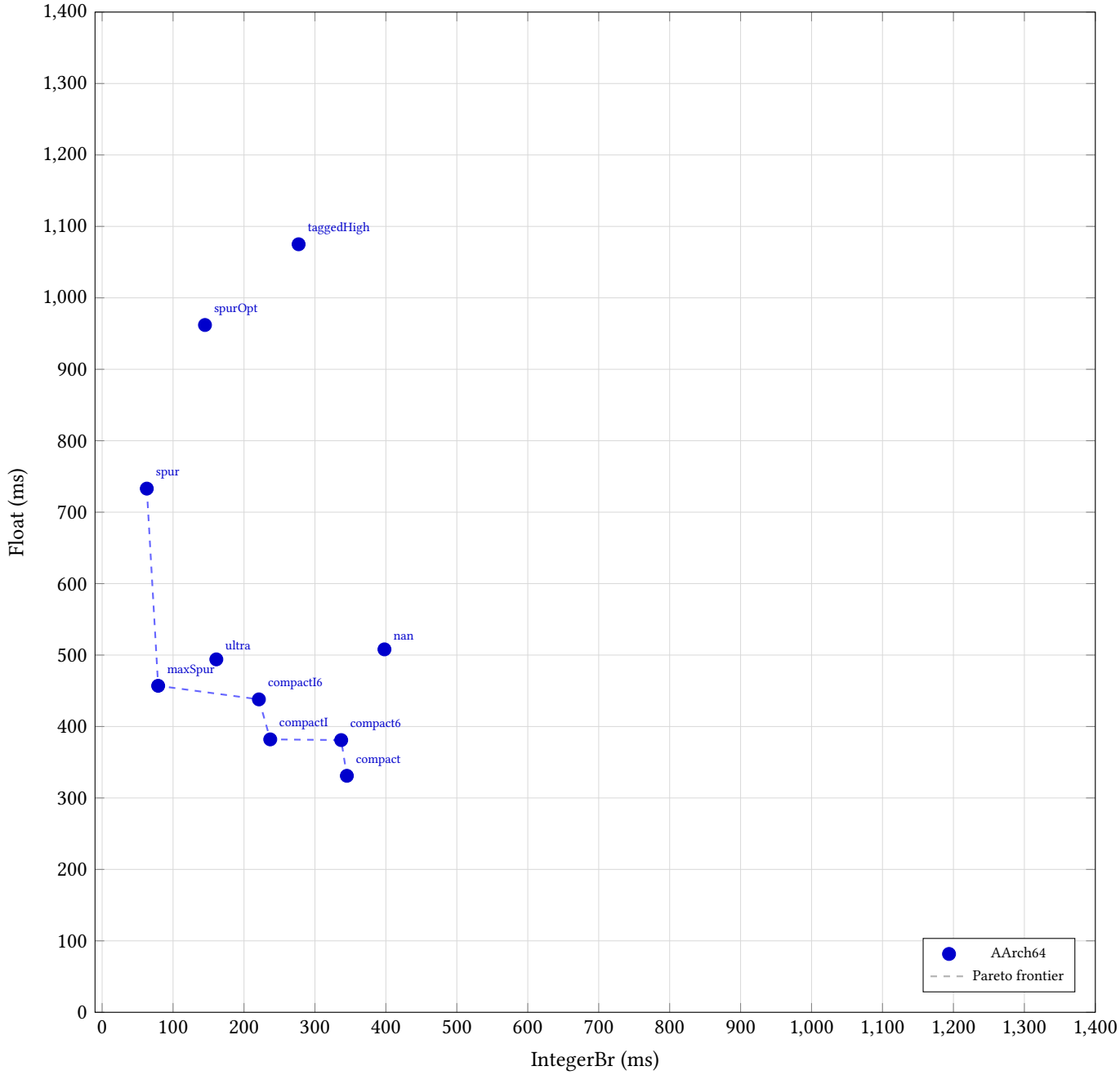


**Figure 3.** IntegerBr vs Float - AARCH64

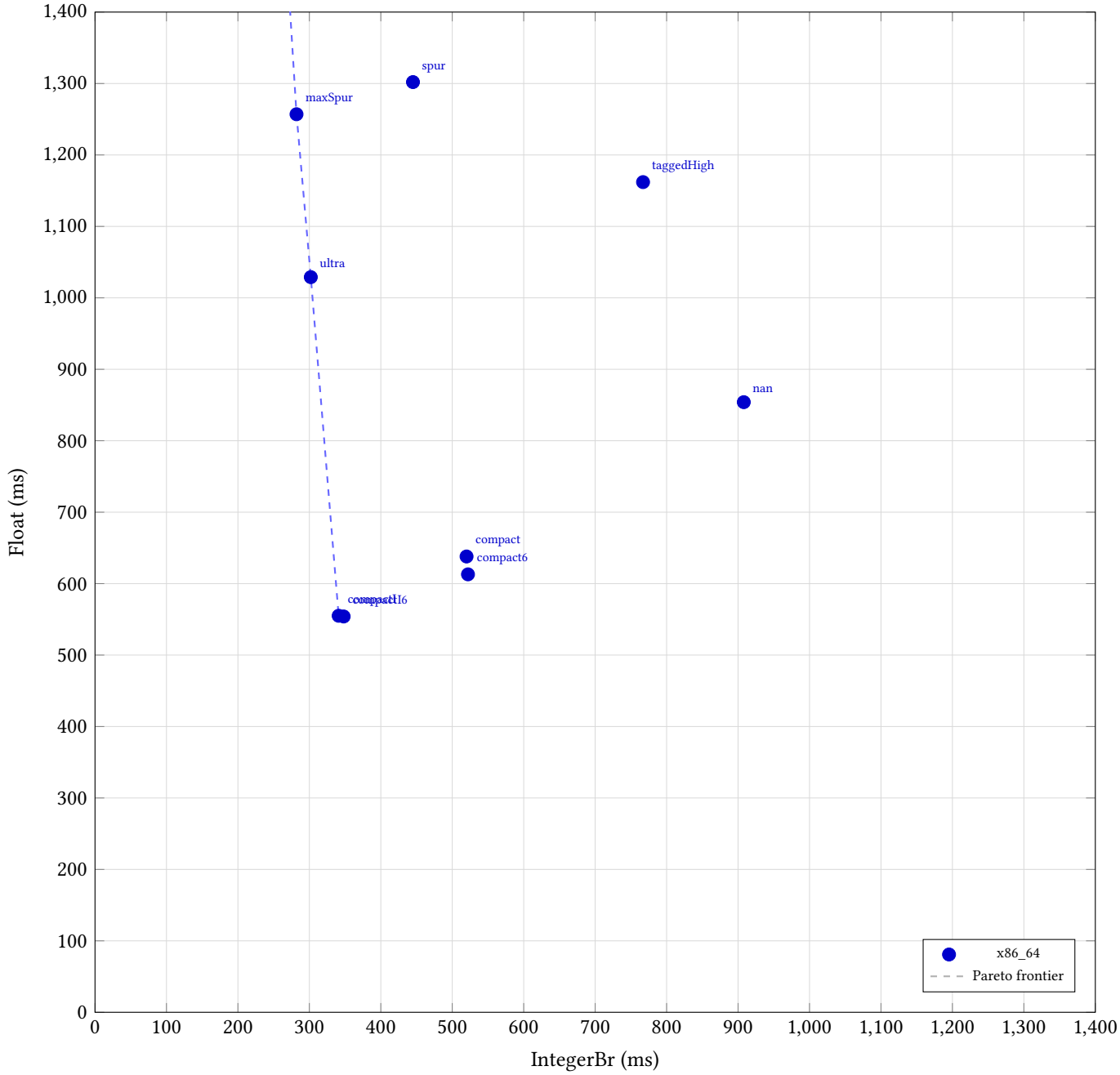


**Figure 4.** IntegerBr vs Float - X86-64

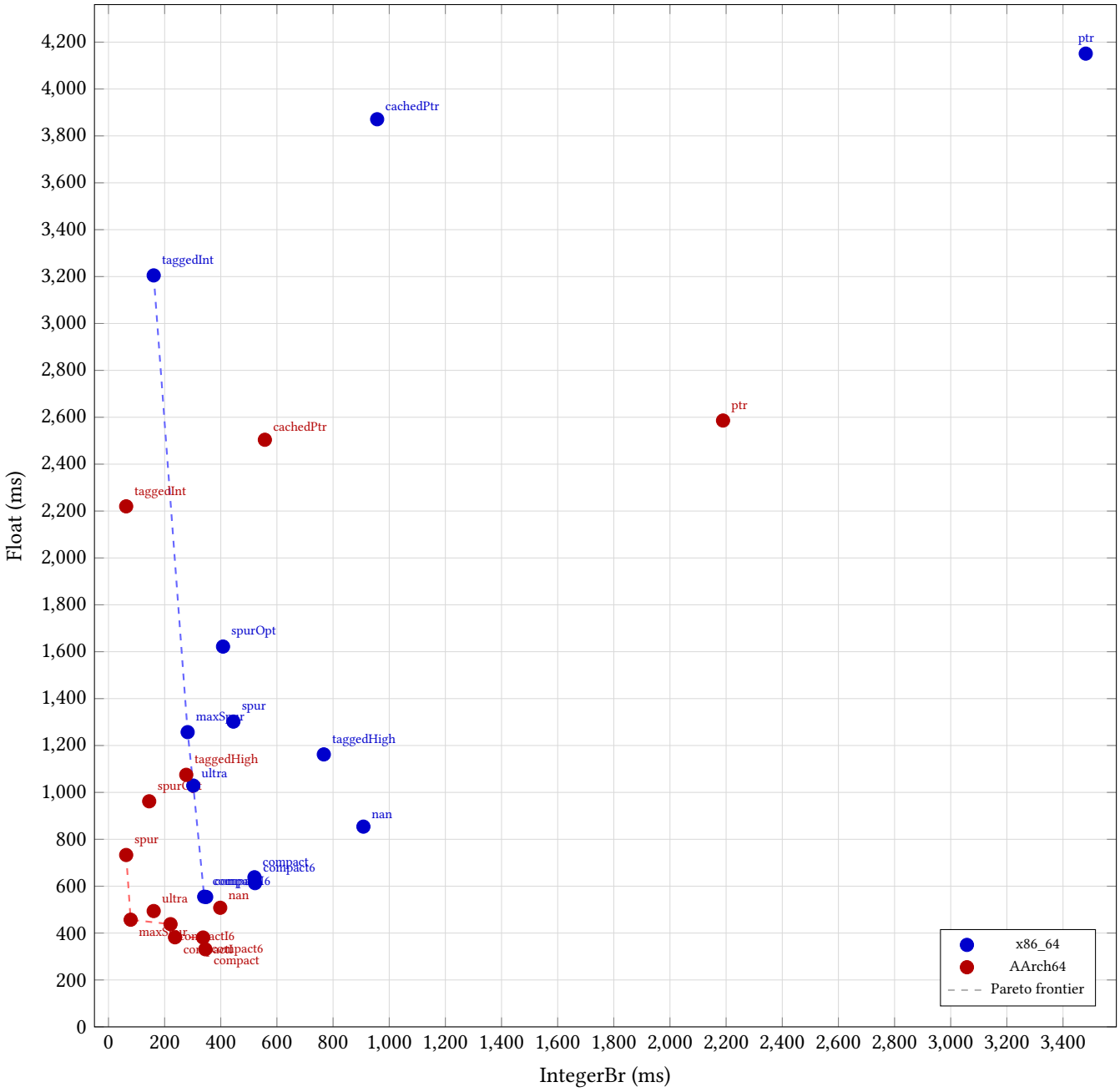


**Figure 5.** IntegerBr vs Float